

“ DIABETES PREDICTION ”

Major Project (CD-805)

Submitted in partial fulfillment of the requirements for the Award of Degree of Bachelor of Technology in Computer Science & Engineering (CSE -DS)

Submitted to



RAJIV GANDHI PROUDYOGIKI VISHWAVIDHYALAYA,

BHOPAL (M.P.)

Submitted by

Abhishek Singh	Enrollment No- 0111CD211003
Anmol Shrivastava	Enrollment No- 0111CD211015
Shubham Ghodeswar	Enrollment No- 0111CD211046
Vikash Kumar	Enrollment No- 0111CD211055

Under the Guidance of

Prof. Prakash Maravi

(CSE -DS Department)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

TECHNOCRATS INSTITUTE OF TECHNOLOGY, BHOPAL (CSE -DS)

SESSION: Jan-June 2025

TECHNOCRATS INSTITUTE OF TECHNOLOGY, BHOPAL
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (CSE -DS)



CERTIFICATE

This is to certify that the work embodies in this Major Project-II (CD-805) work entitled “**Diabetes Prediction**” being submitted by, **Abhishek Singh (Enrolment No.0111CD211003), Anmol Shrivastava(Enrolment No. 0111CD211015), Shubham Ghodeswar (Enrolment No.0111CD211046), Vikash Kumar (Enrolment No.0111CD211055)** in partial fulfillment of the requirement for the award of Bachelor of Technology in COMPUTER SCIENCE & ENGINEERING to Rajiv Gandhi Proudhyogiki Vishwavidyalaya, Bhopal (M.P.) during the academic year 2024-25 is a record of bonafide piece of work, carried out by him/her/them under my supervision and guidance in the Computer Science & Engineering, Technocrats Institute of Technology, Bhopal.

Approved By:

Dr. Divya Jain

Head Of The Department

(Computer Science &

Engineering CSE- DS)

Certified By:

Dr. Shashi Kumar Jain

DIRECTOR

Guided By:

Prof. Prakash Maravi

(Computer Science &

Engineering CSE-DS)

TECHNOCRATS INSTITUTE OF TECHNOLOGY, BHOPAL

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (CSE -DS)



DECLARATION

We , **ABHISHEK SINGH(Enrolment No.0111CD211003)**, **ANMOL SHRISTAVA (Enrolment No. 0111CD211015)**, **SHUBHAM GHODESWAR (Enrolment No.0111CD211046)**, **VIKASH KUMAR (Enrolment No.0111CD211055)** Students Of Bachelor Of Technology in COMPUTER SCIENCE & ENGINEERING(CSE -DS), session 2024-2025 Technocrats Institute of Technology ,Bhopal M.P., hereby declare that the work presented in this project Report entitled “**DIABETES PREDICTION**” is the outcome of our own work, is bona fide and correct to the best of our knowledge and this work has been carried out taking care of Engineering Ethics.

ABHISHEK SINGH	(Enrolment No.0111CD211003)
ANMOL SHRIVASTAVA	(Enrolment No. 0111CD211015)
SHUBHAM GHODESWAR	(Enrolment No.0111CD211046)
VIKASH KUMAR	(Enrolment No.0111CD211055)

ACKNOWLEDGEMENT

Indeed it is a matter of great felicity and privilege for me to take an opportunity to work under the guidance of **(Prof. Neha Jain)** in department of **Computer Science Engineering (& Data Science)** TIT, Bhopal who constantly guided, supported and encouraged at every step of dissertation.

I would like to express my sincere thanks to **Dr. Shashi Jain, Director, TIT, Bhopal** for his kind co-operation and support.

I would also like to acknowledge **Dr. Divya Jain H.O.D, Department of Computer Science Engineering (Data Science)** for his guidance and great cooperation & support.

I would like to express my sincere thanks to, **all Faculty members & Lab In-charges of our Department** for their kind cooperation & constant support.

I am thankful to my parents, friends etc. who helped me in one or other way during the course of my study.

ABHISHEK SINGH	(Enrolment No.0111CD211003)
ANMOL SHRIVASTAVA	(Enrolment No. 0111CD211015)
SHUBHAM GHODESWAR	(Enrolment No.0111CD211046)
VIKASH KUMAR	(Enrolment No.0111CD211055)

Place: TIT, Bhopal

Date : __/__/2025

ABSTRACT

Chest screening procedures that are typically used to detect lung nodules can also be used to diagnose pneumonia, effusion, cardiomegaly, and other disorders. Pneumonia is an infectious and dangerous disease that affects millions of individuals, primarily those over the age of 65 who have chronic disorders such as asthma or diabetes. Chest X-Rays are the most effective approach for determining the amount and location of the infected zone in the lungs during the diagnosis of pneumonia. However, examining chest radio-graphs is not a leisurely task for radiotherapists. This work proposes a novel deep learning approach for detection of pneumonia using deep transfer learning to simplify the detection process with improved accuracy. This work was aimed to preprocess the input chest X-ray images to identify the presence of pneumonia using CNN architectures and classify the X-ray as normal or abnormal (Bacterial/viral pneumonia) using convolutional neural network models such as VGG19, DenseNet201, XceptionNet. Besides, to extract the efficient features and improve the accuracy of pre-trained models, Adam and Stochastic Gradient Descent (SGD) optimizers are used and its performance is analyzed. From the results obtained, it is observed that the proposed model DenseNet201 achieves an test accuracy of 91.49%, which is higher than the other CNN models.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE
NO		NO
	ABSTRACT	i
	LIST OF ABBREVIATIONS	vi
	LIST OF FIGURES	vii
	LIST OF TABLES	viii
1	INTRODUCTION	1
1.1	Objective	3
1.2	Project Report Overview	3
2	LITERATURE SURVEY	4
2.1	Deep Learning	4
2.2	Quality of the data	4
2.3	Image Augmentation	5
2.4	Data Partitioning	5
2.5	Data Pre-processing	6
2.6	Convolutional Neural Networks (CNN)	7
2.7	CNN Models	9
2.7.1	AlexNet Model	9
2.7.2	GoogLeNet Model	10
2.7.3	LeNet Model	10
2.7.4	Strided Net Model	11

2.7.5	RestNet Model	11
2.7.6	VGG19 Net Model	11
2.8	Activation Function	12
2.9	Optimizer	13
2.9.1	Gradient descent	13
2.9.2	Stochastic Gradient Descent	13
2.9.3	Adam (Adaptive Moment Estimation)	14
2.10	Model Parameters	14
2.11	Summary	14
3	METHODOLOGY	15
3.1	System Specification	16
3.1.1	Software Requirements	16
3.1.2	Hardware Requirements	16
3.2	Dataset Preparation	16
3.3	Data Augmentation	17
3.4	CNN overview	18
3.5	Transfer Learning	19
3.6	VGG19	19
3.6.1	Input Layer	20
3.6.2	Convolutional Layer	20
3.6.3	Filter Layer	21
3.6.4	Rectifier Linear Activation Unit	21
3.6.5	Dense Layer	21
3.6.6	Output Layer	21
3.6.7	Model Parameters	22
3.7	Dense Net 201	22
3.7.1	Transition Layer	23
3.8	Xception Net	23

3.8.1	Separable Layers	23
3.8.2	Model Parameters	23
3.9	Training	24
4	RESULTS	26
5	CONCLUSION AND FUTURE WORK	28
5.1	Conclusion	28
5.2	Future Work	28
	REFERENCES	29
	APPENDIX A	32
	APPENDIX B	36
	ONLINE COURSES	38
	PUBLICATION	40

LIST OF FIGURES

FIGURE	TITLE	PAGE
No.		No.
2.1	Simple Convolutional Neural Network Architecture	7
3.1	Proposed method of Pneumonia Detection-Block Diagram	15
3.2	Visualization of Data Augmentation technique - Chest X-Ray	18
3.3	VGG19Architecture Diagram	19
3.4	DenseNet201 Architecture Diagram	22
3.5	XceptionNet Architecture	24
4.1	Accuracy and loss graph of VGG19	26
4.2	Accuracy and loss graph of DenseNet201	26
4.3	Accuracy and loss graph of XceptionNet	27
B.1	Home page (User Interface)	36
B.2	Prediction Result	36
B.3	Test Accuracy for DenseNet201 Model	37
B.4	Test Accuracy for VGG19 Model	37
B.5	Test Accuracy for XceptionNet Model	37

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
•		
3.1	Dataset statistics	17
4.1	Performance Evaluation	27

CHAPTER 1

INTRODUCTION

Pneumonia is a widely occurring severe form of acute respiratory disease that is caused due to infectious agents with high attack rates among individuals, who belong to age groups of either of the extreme ends of the lifespan of an average human being. These infectious agents may be viral, fungal, or bacterial and the lungs react to the infiltration of these foreign microorganisms by an inflammatory response, which causes the bronchioles and alveoli to get filled with liquid. This in turn causes difficulty in respiration for the affected individual [5].

There are three main types of pneumonia: a) community-acquired, b) viral, and c) bacterial. Community-acquired pneumonia is mainly distinguished from nosocomial-acquired pneumonia and roughly 1.5 million people in the US are hospitalized owing to community acquired pneumonia every year. Globally, around 200 million people get affected by viral pneumonia annually, with a 50-50 infection rate between adults and children.

Generally, bacteria are classified as typical or atypical. Typical bacteria can be seen on the Gram stain and cultured through standard media, which, on the other hand, is not seen in the atypical class of bacteria. Typical bacteria-caused pneumonia can be listed as staphylococcus aureus, streptococcus pneumonia, Haemophiles influenza, etc. whereas atypical bacteria-caused pneumonia is mostly caused by chlamydia pneumonia, legionella, chlamydia psittaci, and mycoplasma.

Around 450 million people worldwide are affected by pneumonia and result in about four million deaths each year [10]. Pneumonia is the most serious illness in children younger than 5 years of age [9]. India, with 158,176 deaths in 2016, continues to have the highest number of pneumonia infant deaths in the world. The report, released on World Pneumonia Day, found that by 2030 nearly 11 million children under five were likely to be killed by the infectious disease.

In the 19th century, William Osler considered pneumonia to be "the captain of the men of death."

Physical examination, medical history, clinical examinations such as sputum, chest Xrays, and some other imaging techniques are how doctors diagnose pneumonia in hospital patients. Chest X-rays, which is now becoming less costly due to technological advances in biomedical equipment, is the most common technique used for detection of pulmonary diseases such as pneumonia. The problem of non-availability of experts can be solved by computer aided

diagnosis. Current development in artificial intelligence can be useful in the medical field. The study of Deep Learning can be used to classify chest X-ray images to detect pneumonia.

Deep learning is a machine learning technique that teaches computers to do what comes naturally to humans: learn by example. Deep learning is a key technology behind driverless cars, enabling them to recognize a stop sign or to distinguish a pedestrian from a lamppost.

Deep Learning has proved to be a very powerful tool in the past few decades because of its ability to handle large amounts of data. The interest in using hidden layers has surpassed traditional techniques, especially pattern recognition. One of the most popular deep neural networks is the Convolutional Neural network.

Since the 1950s, the early days of AI, researchers have struggled to make a system that can understand visual data. In the following years, this field came to be known as Computer Vision. In 2012, computer vision took a quantum leap when a group of researchers from the University of Toronto developed an AI model that surpassed the best image recognition algorithms and that too by a large margin. The AI system, which became known as AlexNet (named after its main creator, Alex Krizhevsky), won the 2012 ImageNet computer vision contest with an amazing 85 percent accuracy. The runner-up scored a modest 74 percent on the test.

At the heart of AlexNet were Convolutional Neural Networks, a special neural network that roughly imitates human vision. Over the years CNNs have become a very important part of many Computer Vision applications.

In deep learning, a convolutional neural network is a class of deep neural networks, most commonly applied to analyse visual imagery. It uses a special technique called Convolution. Convolution is a mathematical operation on two functions that produces a third function that expresses how the shape of one is modified by the other.

CNNs were first developed and used around the 1980s. The most that a CNN could do at that time was recognize handwritten digits. It was mostly used in the postal sectors to read

zip codes, pin codes, etc. The important thing to remember about any deep learning model is that it requires a large amount of data to train and also requires a lot of computing resources. This was a major drawback for CNNs at that period and hence CNNs were only limited to the postal sectors and it failed to enter the world of machine learning.

Convolutional neural networks are composed of multiple layers of artificial neurons. Artificial neurons, a rough imitation of their biological counterparts, are mathematical functions that calculate the weighted sum of multiple inputs and output an activation value. When an image is fed to a ConvNet, each layer generates several activation functions that are passed on to the next layer.

The first layer usually extracts basic features such as horizontal or diagonal edges. This output is passed on to the next layer which detects more complex features such as corners or combinational edges. Deeper layers can identify even more complex features such as objects, faces, etc.

Similar to the Convolutional Layer, the Pooling layer is responsible for reducing the spatial size of the Convolved Feature. This is to decrease the computational power required to process the data by reducing the dimensions. There are two types of pooling average pooling and max pooling.

1.1 Objective

The objective of this work is to build a robust convolutional neural network model for pneumonia detection using chest X-ray images to provide a fast and accurate diagnosis of pneumonia.

1.2 Project Report Overview

Chapter 1 provides a brief overview of challenges in pneumonia detection and deep learning. In Chapter 2, existing systems have been explored along with the research done by various other developers and scientists in the field of Convolutional Neural Networks Algorithms and CNN architectures. Chapter 3 describes the methodology for the pneumonia disease prediction from chest X-ray images. Chapter 4 describes the result and chapter 5 outlines the further development of the project.

CHAPTER 2 LITERATURE SURVEY

Medical images utilizing CAD systems has been used for decades ever since the age of artificial intelligence. Since then AI technology has been elevated to machine learning and to

deep learning that resulted in an improved acceleration through its performance. This developed technology turns to different techniques like classifying diseases of human-related, animals, and plant diseases. With an increased quantity of datasets, CNN has been the highly recommended technique for identifying and classifying numerous types of medical-related images.

2.1 Deep Learning

Learning from the Deep Learning point of view is a method of cultivating the actions based on knowledge and experience. Deep Learning is an artificial intelligence and machine learning subgroup which expands the output of countless machine learning applications like AI. Deep architecture technique organized with many processing parts that started in the early 1960s has affected non-linearity and computing power based on the GPU via acceleration, allowing for deeper networks that have better used their resources [9]. Deep architectures composed of numerous hidden-layers improve the neural network's overall performance. Since the 1980s and up to date the invention of back-propagation is still utilized to preserve the neural network. The authors in have developed numerous models to confirm an accurate result of the top model in detecting pneumonia. Their study trained various types of CNN models namely AlexNet, LeNet, GoogleNet, ResNet, and VGGNet of using a 226x226 image resolution with a dataset of 26,684 images. The result of their study reached 92% accuracy rate for VGGNet and 50% is the lowest rate attained by the ResNet model[4].

2.2 Quality of the Data

Deep learning is applied to unstructured data like images, video, sound or text. An image is just a blob of pixels, a message is just a blob of text. This data is not organized in a typical, relational database by rows and columns. So, once the data is collected, they are pre-processed into a format the deep learning algorithm can use for the model. Following are some of the problems that can arise in data collection such as collected data could be unrelated to the problem statement. Data may be imbalanced. Some classes or categories in the data may have a disproportionately high or low number of corresponding samples[5]. As a result, they risk being under-represented in the model, and the size of a dataset affects the memory and processing required for iterations during training. Normalization reduces the size by reducing the order and magnitude of data.

2.3 Image Augmentation

Deep learning models usually require enormous amounts of data to train for them to correctly function. In the case of CNNs, thousands upon thousands of images are required which, pragmatically speaking, is a difficult prospect. To solve this, a technique named image augmentation is used [3]. Image augmentation virtually increases the size of an existing dataset to a large extent with techniques like standardization of features (pixel values), whitening transforms [5], random rotations and shifts, flipping, re scaling, shearing, zooming, etc. The Author have rescaled, shear, zoomed and flipped horizontally the training image for a wider variety and augmentation of training data.

Image segmentation is the process of partitioning a digital image into multiple image segments, also known as image regions or image objects (sets of pixels). The goal of segmentation is to simplify and/or change the representation of an image into something that is more meaningful and easier to analyse [10].

Image segmentation is typically used to locate objects and boundaries (lines, curves, etc.) in images. More precisely, image segmentation is the process of assigning a label to every pixel in an image such that pixels with the same label share certain characteristics.

2.4 Data partitioning

Data partitioning in data mining is the division of the whole data available into two or three non-overlapping sets: the training set, the validation set, and the test set. If the data set is very large, often only a portion of it is selected for the partitions [2]. Partitioning is normally used when the model for the data at hand is being chosen from a broad set of models. The basic idea of data partitioning is to keep a subset of available data out of analysis, and to use it later for verification of the model.

Data splitting is the act of partitioning available data into two portions, usually for cross validation purposes. One portion of the data is used to develop a predictive model, and the other to evaluate the model's performance[7]. It takes the chest- X ray images as input and divides the data set into practice (98637 images, 28744 patients), validation (6351 images, 1672 Patients), and testing (420 images, 389 patients) for training the model. Data might be split in as 80-20 or a 67-33 ratio of training, testing data.

Before inputting the images into the network, the images have been resized to 224 x 224 and normalized it. The system produces the likelihood of pneumonia including a heat-map that locates the most likely area of pneumonia.

2.5 Data pre-processing

Data partitioning in data mining is the division of the whole data available into two or three non-overlapping sets, the training set, the validation set, and the test set. If the data set is very large, often only a portion of it is selected for the partitions[8]. Partitioning is normally used when the model for the data at hand is being chosen from a broad set of models. To build useful Deep Learning models, the validation error must continue to decrease with the training error. Data Augmentation is a very powerful method of achieving this. The augmented data will represent a more comprehensive set of possible data points, thus minimizing the distance between the training and validation set, as well as any future testing sets. Data Augmentation is the technique developed to reduce overfitting. This technique enables practitioners to significantly increase the diversity of data available for training models, without actually collecting new data and improve performance. Many other strategies for increasing generalization performance focus on the model's architecture itself. This has led to a sequence of progressively more complex architectures from AlexNet, VGG-16, ResNet, Inception-V3 and Dense Net. Functional solutions such as dropout regularization, batch normalization, transfer learning, and pre-training have been developed to try to extend Deep Learning for application on smaller datasets.

2.6 Convolutional Neural Network

A Convolutional Neural Network (ConvNet/CNN) is a Deep Learning algorithm which can take in an input image, assign importance (learnable weights and biases) to various

aspects/objects in the image and be able to differentiate one from the other stimulated by the human brain. The convolutional neural network is a feed-forward neural network[2]. It is capable of extracting characterized features and classifies them. The standard convolutional layer acquires image input that creates map features through the kernel. It works by taking each neuron from the preceding layer. Increased performance of improved illustration of images is achieved because of multiple neuron connections and overlapping with each other. Weight sharing reduces the number of parameters. Every connected convolutional layer to non-linear activation layer accelerates CNN's conventional architecture as illustrated in Figure 2.1

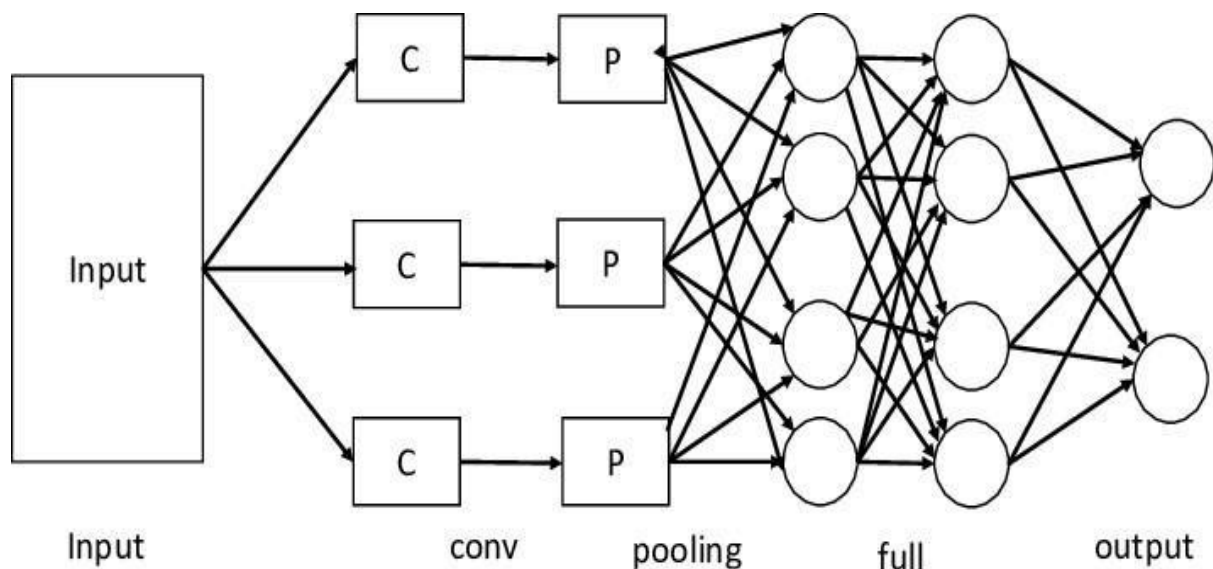


Figure 2.1 Simple Convolutional Neural Network Architecture Diagram

Applying various convolutional filters, CNN machine learning models can capture the high-level representation of the input data, there are different kinds of models available in the CNN such as LeNet, AlexNet, GoogLeNet, ResNet, etc.

Numerous authors have been using several CNN models in their research. The plain and simple architecture was developed using the LeNet model was introduced in the works of Le Cun et.al. They modified layers, kernels, optimizers, and hyper-parameters to train their model. Their proposed model, composed of several kernels in one convolution learns extra complex functions that compute multiple map features with activation and pooling layers[10].

On ResNet18, Xception, InceptionV3, DenseNet121, and MobileNetv3, Hashmi presented a state-of-the-art based deep learning technique where certain pre-trained convolutional architectures and weighted classifier are utilized for transfer learning utilizing fine-tuning. There were also some partial data augmentation approaches utilized. Finally, using a weighted classifier, data from the Guangzhou Women and Children's Medical Centre pneumonia dataset yielded a testing accuracy of 82.43 percent .

- The Author's work is based on the well-established pre trained DL model (VGG19) and the attention module.

- VGG-16 helps to overcome the over fitting problem as there are limited amount of CXR images for training purpose. The proposed method (also called Attentionbased VGG16) consists of four main building blocks such as Attention module, Convolution module, FC layers, and Soft max classifier [14].

- The scale-invariant convolution module captures the interesting clues of the image. The interesting clues are extracted from the mid level layer that is more appropriate to CXR images. However, the features from other layers (higher or lower) are not appropriate to CXR images because such images are neither more general nor more specific.

The standard convolutional layer acquires image input that creates map features through the kernel. It works by taking each neuron from the preceding layer. Increased performance of improved illustration of images because of multiple neuron connections and overlapping with each other [15]. Weight sharing reduces the number of parameters. Every connected convolutional layer to nonlinear activation layer accelerates CNN in more complex functions that may lead to overfitting and pooling is used to overcome the overfitting and also reduce the number of parameters. Fully-connected layers summarize the trained features to provide the classified result.

The limitation is that despite the power and resource complexity of CNNs, they provide in-depth results. At the root of it all, it is just recognizing patterns and details that are so minute and inconspicuous that it goes unnoticed to the human eye. But when it comes to understanding the contents of an image it fails.

These limitations are more than evident when it comes to practical applications. For example, CNNs were widely used to moderate content on social media. But despite the vast resources of images and videos that they were trained on it still is not able to completely block and remove inappropriate content.

Several studies have shown that CNNs trained on ImageNet and other popular datasets fail to detect objects when they see them under different lighting conditions and from new angles. Despite the limits of convolutional neural networks, however, there's no denying that they have caused a revolution in artificial intelligence. Today, CNNs are used in many computer vision applications such as facial recognition, image search, and editing, augmented reality, and more. As advances in convolutional neural networks show, these achievements are remarkable and useful, but still very far from replicating the key components of human intelligence [7].

2.7 CNN Models

This sub section describes the use of CNN architectures for modeling pneumonia disease detection.

2.7.1 AlexNet model

AlexNet which was developed by A.Krizhevsky for visual object recognition in 2012 won an ImageNet Challenge. It is the most studied architecture of CNN for classification tasks on images [11]. AlexNet structure has 5 convolutional layers, 2 fully-connected layers with max pooling, and the activation layer that connects to 1000 classes. Its parameters comprise of 61 million values. It has eight layers with learnable parameters [6]. The model consists of five layers with a combination of max pooling followed by 3 fully connected layers and they use Relu activation in each of these layers except the output layer.

ReLU Nonlinearity -- AlexNet uses Rectified Linear Units (ReLU) instead of the tanh function, which was standard at the time. ReLU's advantage is in training time; a CNN using ReLU was able to reach a 25% error on the CIFAR-10 dataset six times faster than a CNN using tanh function.

Multiple GPUs -- AlexNet allows for multi-GPU training by putting half of the model's neurons on one GPU and the other half on another GPU. Not only does this mean that a bigger model can be trained, but it also cuts down on the training time.

Overlapping Pooling -- CNNs traditionally “pool” outputs of neighboring groups of neurons with no overlapping. However, when the authors introduced overlap, they saw a reduction in error by about 0.5% and found that models with overlapping pooling generally find it harder to overfit [12].

2.7.2 GoogLeNet model

GoogLeNet architecture's goal is to learn with its parameters reduced. It is inspired based on Inception architecture with a total of 22 convolutional layers using small convolutions and small - batch normalizations. Its parameters comprise of 4 million values[19].

The GoogLeNet architecture proposed by Szegedy is 22-layer deep network consisting of inception modules, instead of uniformly progressive layers. An inception block accommodates a large number of units at each stage by hosting parallel convolution and pooling layers, resulting in an uncontrolled computational complexity because of the increased number of parameters. To control the computational complexity, the GoogLeNet model uses inception blocks with dimension reduction, rather than the naive inception block. The performance of GoogLeNet, in which the inception block was introduced, proves that an optimal sparse architecture built from the available dense building blocks improves the performance of artificial neural networks for computer vision tasks.

2.7.3 LeNet model

LeNet-5 was developed by one of the pioneers of deep learning, Yann LeCun in 1998 in his work titled ‘Gradient-Based Learning Applied to Document Recognition’. LeNet was used in detecting handwritten cheques by banks based on MNIST dataset. Fully connected networks and activation functions were previously known in neural networks. LeNet-5 introduced convolutional and pooling layers. LeNet-5 is believed to be the base for all other ConvNets. LeNet architecture comprises of 2 convolutional, pooling layers with a flattened convolutional layer, 2 fully-connected layers, and SoftMax classifier. LeNet is straightforward and small, making it perfect for teaching the basics of CNNs, it can run on the central processing unit (CPU) even if the system does not have a suitable graphical processing unit (GPU).

LeNet's parameter consists of 60 thousand values only [13].

2.7.4 StridedNet model

StridedNet architecture employs stride convolutions for lower volume size, the use of batch normalization, and hyperparameters fine-tuning to improve training accuracy. The utilization of dropout is to help the network generalize and minimize overfitting. The stridedNet parameters are approximately 1 million [20].

2.7.5 ResNet-50 model

ResNet consists of 3 convolutions for a total of 75 layers deep in every convolution block and combines several sizes of convolution kernels with residual blocks during training to lessen training time. Its parameters comprise of 61 million values.

The ResNet-18 model proposed by He et al. It is based on a residual learning framework, which increases the efficiency of deep network training. The residual blocks in the ResNet models facilitate the optimization of the overall network, which in turn improves model accuracy, unlike the original unreferenced mapping in monotonically progressive convolutions. These residuals or “skip connections” perform identity mapping, which neither adds parameters nor increases the computational complexity [17].

2.7.6 VGGNet-16 model

Simonyan and Zisserman introduced VGG network architecture in 2014. VGG-16 architecture consists of 16 convolutional layers, 2 fully-connected layers with 4,096 nodes for each layer. Its parameters comprise of 138 million values and it is considered the most used image-detection architectures.

The input to the cov1 layer is a fixed size 224 x 224 RGB image. The image is passed through a stack of convolutional (conv.) layers, where the filters were used with a very small receptive field: 3×3 [16]. In one of the configurations, it also utilizes 1×1 convolution filters, which can be seen as a linear transformation of the input channels. The convolution stride is fixed to 1 pixel; the spatial padding of Conv. layer input is such that the spatial resolution is preserved after convolution, i.e. the padding is 1-pixel for 3×3 Conv. layers. Spatial pooling is carried out by five max-pooling layers, which follow some of the Conv. layers (not all the conv.

layers are followed by max-pooling). Max-pooling is performed over a 2×2 pixel window, with stride 2.

Three Fully-Connected (FC) layers follow a stack of convolutional layers (which has a different depth in different architectures): the first two have 4096 channels each, the third performs 1000way ILSVRC classification and thus contains 1000 channels (one for each class). The final layer is the soft-max layer. The configuration of the fully connected layers is the same in all networks.

All hidden layers are equipped with the rectification (ReLU) non-linearity. It is also noted that none of the networks (except for one) contain Local Response Normalisation (LRN), such normalization does not improve the performance on the ILSVRC dataset, but leads to increased memory consumption and computation time[18].

2.8 Activation Function

Activation function Relu is used throughout except for the last layer where Soft max is used for binary classification problem. Activation Function decides whether a neuron should be activated or not. The choice of activation function in the hidden layer will control how well the network model learns the training dataset. The choice of activation function in the output layer will define the type of predictions the model can make. The input is fed to the input layer, the neurons perform a linear transformation on this input using the weights and biases, according to the following equation :

$$x = (\text{weight} * \text{input}) + \text{bias} \text{ ----- } 2.1$$

The rectified linear activation is the default activation when developing multilayer Perceptron and convolutional neural networks. But there are different types of activation function used in model suchs as Sigmoid tanh Activation Function, Leaky ReLU, Softmax Activation Function. Neural network without any activation function would not be able to realize such complex mappings mathematically and would not be able to solve real world tasks.

2.9 Optimizers

Optimizers are algorithms or methods used to minimize an error function(loss function)or to maximize the efficiency of production. Optimizers are mathematical functions which are dependent on model's learnable parameters i.e Weights & Biases [23]. The loss function is the core of deep learning. Simply, it is a method of evaluating how good the model is predicting. Optimizers are methods used to change the attributes of neural network such as weights and learning rate in order to reduce the losses. Categorical crossentropy is a loss function that is used in multi-class classification tasks. These are tasks where an example can only belong to one out of many possible categories, and the model must decide which one. Formally, it is designed to quantify the difference between two probability distributions[20]. Gradient descent is a first-order optimization algorithm which is dependent on the first order derivative of a loss function. It calculates which way the weights should be altered so that the function can reach a minima. Through back propagation, the loss is transferred from one layer to another and the model's parameters also known as weights are modified depending on the losses so that the loss can be minimized [21]. So it has huge drawback of requiring large memory to calculate gradient on the whole dataset. Adam is an optimization algorithm that can be used instead of the classical stochastic gradient descent procedure to update network weights iteratively based in training data.

2.9.1 Gradient Descent

Gradient descent is an optimization algorithm based on a convex function and tweaks its parameters iteratively to minimize a given function to its local minimum. Gradient Descent iteratively reduces a loss function by moving in the direction opposite to that of steepest ascent. It is dependent on the derivatives of the loss function for finding minima. uses the data of the entire training set to calculate the gradient of the cost function to the parameters which requires large amount of memory and slows down the process [22].

2.9.2 Stochastic Gradient Descent

SCD is a variant of Gradient Descent. It update the model parameters one by one. If the model has 10K dataset SGD will update the model parameters 10k times. Batch gradient descent performs redundant computations for large datasets, as it recomputes gradients for similar examples before each parameter update.

SGD does away with this redundancy by performing one update at a time. It is therefore usually much faster and can also be used to learn online. SGD performs frequent updates with a high variance that cause the objective function to fluctuate heavily as in Image 1.[21]

2.9.3 Adam(Adaptive Moment Estimation)

Adam optimizer is one of the most popular and famous gradient descent optimization algorithms. It is a method that computes adaptive learning rates for each parameter. It stores both the decaying average of the past gradients, similar to momentum and also the decaying average of the past squared gradients, similar to RMS-Prop and Adadelta. Thus, it combines the advantages of both the methods.[24]

2.10 Model Parameters

Deep learning often deals with large amounts of data. This data is broken down into smaller chunks called batches and fed to the neural networks one-by-one. The batch size is the total number of training examples in a batch. Gradient descent is an iterative process and updating the parameters through back propagation in a single pass is not enough. One epoch is when the **entire** dataset is passed forward and backward through the neural network once. Epoch is once all images are processed one time individually of forward and backward to the network, then that is one epoch. I like to make sure my definition of epoch is correct.

2.11 Summary

From the study of literature, it is observed that there are different types of architectures in CNN. Each architecture has specific types of layer that increases the complexity and performance of the model. Data augmentation parameters are relevant to the given Dataset. The model performance can be improved by adding layers in the pre-trained models. The parameters such as loss function, epoch, batch size, optimize are used to tune pre-trained models.

CHAPTER 3

METHODOLOGY

Deep learning models can interpret medical images like X-ray, MRI scan, CT scan to perform diagnosis. The algorithms can detect any risk and flag anomalies in the medical images. Deep learning is extensively used in detecting cancer. The reuse of a previously learned model on a new problem is known as transfer learning, particularly in deep learning since it can train deep neural networks with a large amount of data. In the proposed method, Chest X-Ray images are collected, and the images in the dataset are pre-processed in various CNN architectures that have been utilized in this study to detect Pneumonia or Normal

X-rays. The block diagram of the system is shown in Figure 3.1

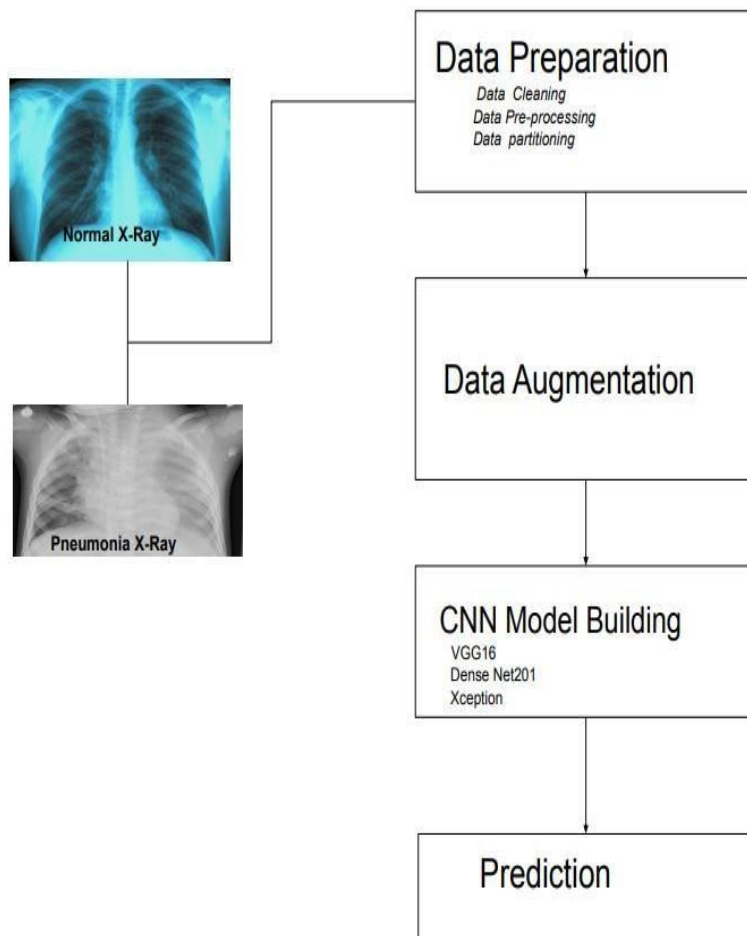


Figure 3.1 Proposed method of Pneumonia Detection-Block Diagram

3.1 System Specification

Following are the hardware and software specifications using which this project was developed.

3.1.1 Software Requirements

Environment : Google Collaboratory

Programming Language: Python 3.7

Processing Unit : Tensor flow Processing Unit

3.1.2 Hardware Requirements

RAM: 16 GB

Processor: Intel i5 Hexi Core processors

3.2 Dataset Preparation

Dataset used was Mendeley Dataset V2 having 5856 Chest X-ray images from both Pneumonia and Normal class. In the Pneumonia dataset, both the combination of viral and bacterial pneumonia x-ray images is stored. Before proceeding with the proposed method, the pictures were partitioned into training and testing sets.

In this work, the dataset extracted by Mendeley website having 5841 Chest X-ray images of both Pneumonia and Normal class had been used. In the Pneumonia dataset, both the combination of Viral and Bacterial Pneumonia x-ray images are stored. The images were partitioned into training and testing sets in the ratio of 66/33. A total of 1929 pictures were set apart for testing and 3912 images were used for training. All the images in the dataset are in 512*512 pixels in size. The number of images in training and testing in both classes is listed in Table 3.1.

Table 3.1 Dataset Statistics

Data	Normal	Viral Pneumonia	Bacterial Pneumonia
Training Set	1055	1000	1857
Testing Set	520	494	915
Total	1575	1494	2772

The dataset is organized into 3 section train, test, validation and contains sub folders for each image category, Pneumonia and Normal. Web scraping is used to extract the minor pneumonia chest x-ray images from the websites for the better performance of the model. The data is mounted into the Google drive for fast processing of the large volume of data. Once the dataset is uploaded into the Google drive there is no need to spend network data every time.

Chest X-ray images were selected from retrospective cohorts of pediatric patients from Guangzhou Women and Children's Medical Centre, Guangzhou. All chest X-ray imaging was performed as part of patient routine clinical care. For the analysis of chest xray images, all chest radiographs were initially screened for quality control by removing all low quality or unreadable scans. The diagnoses for the images were then graded by two expert physicians before being cleared for training the AI system. In order to account for any grading errors, the evaluation set was also checked by a third expert.

3.3 Data Augmentation

Data augmentation techniques are used to increase the input images and also resize the images of dataset from 512*512 converted into 224*224 pixels to reduce the heavy computation and for faster processing.

The resized images are then transformed from RGB to grey scale images. Image data generator function is used to perform augmentation. Augmentation involves image rotation, zooming, width shifting, height shifting, horizontal flip and image contrast enhancement.Keras

Image Data Generator class provides an easy way to augment the images. The keras image data generator class is designed to provide real-time data augmentation and it requires lower memory usage. It generates augmented images on the fly while the model is still in the training stage. Image data generator class ensures that the model receives new variations of the images at each epoch, but it only returns the transformed images and does not add it to the original corpus of images. The sample images of data augmentation techniques are shown in Figure

3.2.

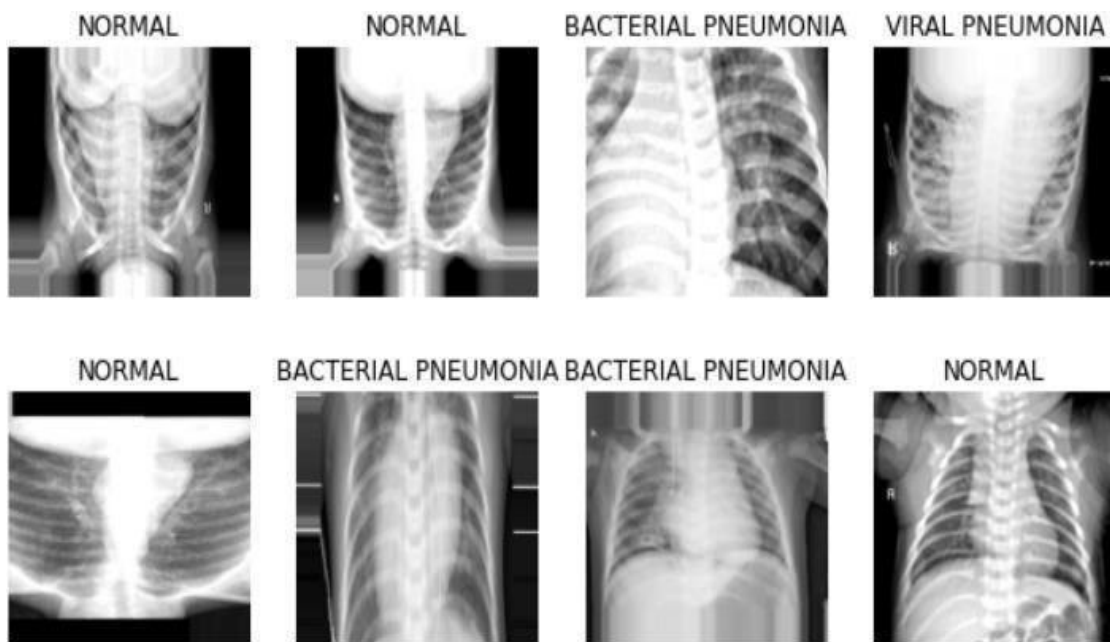


Figure 3.2 Visualization of data augmentation techniques of Chest X ray

3.4 CNN Overview

A convolutional neural network is a network architecture for deep learning which learns directly from data, eliminating the need for manual feature extraction. CNNs are particularly useful for finding patterns in images to recognize objects, faces, and scenes. They can also be quite effective for classifying non-image data such as audio, time series, and signal data. CNN can have tens or hundreds of layers that each learn to detect different features of an image. Filters are applied to each training image at different resolutions, and the output of each convolved image is used as the input to the next layer. The filters can start

as very simple features, such as brightness and edges, and increase in complexity to features that uniquely define the object.

3.5 Transfer Learning

Transfer learning is the reuse of a pre-trained model on a new problem. It's currently very popular in deep learning because it can train deep neural networks with comparatively little data. This is very useful in the data science field since most real-world problems typically do not have millions of labelled data points to train such complex models. Transfer learning (TL) is a research problem that focuses on storing knowledge gained while solving one problem and applying it to a different but related problem.

3.6 Visual Geometry Group19Architecture

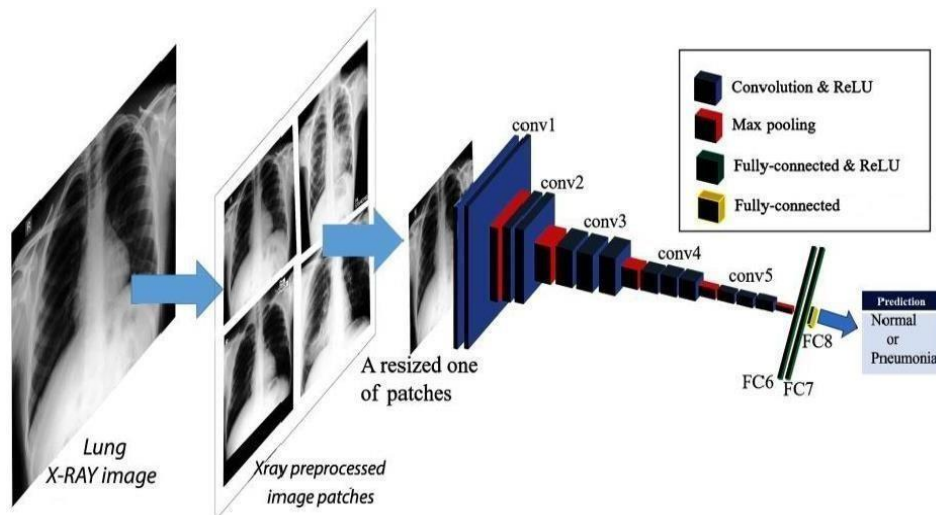


Figure 3.3 VGG19 Architecture Diagram [Source[1]]

VGG19 with weights trained on the ImageNet database is used for extracting features from the images. ImageNet database consists of more than 14 million images categorized into 1000 classes. Since pre-trained models like VGG19 have already learned to extract features from the images, these models have shown magnificent performance when applied on datasets of similar domains. The network consisted of 16-layers with 3 convolutional layers, which are followed by 5 dense layers and the output layer. For the dimensionality reduction, a series of max-pooling layers have been introduced in the architecture VGG19 is having a large number of hyper-parameters. It has convolution layers of 3x3 filter size with a stride of 1 and always uses same padding and max pool layer of 2x2 filter size with stride

of 2. It follows this arrangement of convolution and max pool layers consistently throughout the whole architecture. In the end it has 2 fully connected layers followed by a soft max function for output. The 16 in VGG19 refers to the 16 layers that have weights. This network is a pretty large network and it has about 138 million parameters. Following each convolutional layer, a batch normalization layer is used to normalize the data that comes from the convolutional layer, allowing for less complex processing after the convolutional layer. A pooling layer is utilized with each convolutional layer. The purpose of the pooling layers is to make the feature map smaller. There are several types of pooling layers, with max pooling being the most effective and widely utilized. This pooling layer takes the maximum feature portion from the images and makes a smaller representation. They are often applied after the convolutional layer in order to decrease the complexity of the calculation.

3.6.1 Input Layer

The input layer in CNN should contain image data. Image data is represented by a three-dimensional matrix and it has to be reshaped into a single column. The pre-processed grey images of the dataset have a size of 224 * 224 pixels and the images are fed into the model layers.

3.6.2 Convolution Layer

The main task of these layers is to find out the similarity or connection of features from the previous layer. These layers then make a feature map using the edges of the features. In this proposed method, six convolutional 2D layers are used along with some filters to get the feature maps. These feature maps represent features or edges from the images using the filters to assemble them. Every filter is trained to obtain the edges or features from the Chest X-ray images from the dataset. The output feature map is obtained using the following equation

$$M_x^l = B_x^l + \sum_{y=1}^{k^{l-1}} F_{x,y}^l * M_y^{l-1} \quad \text{..... (3.1)}$$

Here, B_x^l represents the bias, l means layers, $F_{x,y}^l$ represents filter, k^{l-1} denotes the number of filters, M_y^{l-1} and M_x^l represents the input feature map and output feature map.

3.6.3 Filter layer

Filters detect patterns such as edges in an image by detecting the changes in intensity values of the image. In terms of an image, a high-frequency image is the one where the intensity of the pixels changes by a large amount, whereas a low-frequency image is the one where the intensity is almost uniform. Usually, an image has both high and low frequency components. The high-frequency components correspond to the edges of an object because at the edges the rate of change of intensity of pixel values is high.

3.6.4 Rectified linear activation unit

After each convolution operation, RELU a non-linear activation function is applied. This operation is applied to each pixel and replaces all the negative pixel values in the feature map with zero. Usually, the image is highly non-linear, which means it has varied pixel values. In this scenario it is very difficult for an algorithm to make correct predictions. RELU activation function is applied in these cases to decrease the nonlinearity and make the job easier. Therefore this layer helps in the detection of features, decreasing the non-linearity of the image, converting negative pixels to zero which also allows detecting the variations of features.

3.6.5 Dense Layers

Dense Layer is simple layer of neurons in which each neuron receives input from all the neurons of previous layer, thus called as dense. Dense Layer is used to classify image based on output from convolutional layers. The input to the fully connected layer is the output from the final Pooling or Convolutional Layer, which is flattened and then fed into the fully connected layer.

3.6.6 Output Layer

For classification purposes, a soft max activation function is employed in the output layer of the algorithm. It is referred to as the ReLu activation function. It is mainly used for finding the probability prediction using the values from 0 to 1. The problem consists of binary classification where input is an image, and the classifier gives output either as "0" (Nonpneumonia) or "1" (Pneumonia)

3.6.7 Model Parameters

The total parameters of the VGG19 architecture are 134,268,738 and all of them are learnable parameters. All the Max-Pooling layers lack parameters, and the batch normalization layers do not have any parameters that can be learned from their predecessors.

3.7 DenseNet201

DenseNet201, which is a short form of the Dense Convolutional Network, needs less numbers of parameters than a conventional CNN. DenseNet has four different variants, DenseNet121, DenseNet169, DenseNet201, and DenseNet264. In DenseNet201 architecture, each layer is connected to every other layer. The feature maps of all the preceding layers are used as inputs. The input of a layer inside DenseNet201 is the concatenation of feature maps from previous layers. DenseNet201 require fewer parameters than an equivalent traditional CNN, as there is no need to learn redundant feature maps. DenseNet201 architecture is shown in figure 3.4. Dense nets layers are very narrow (e.g. 12 filters), and they just add a small set of new featuremaps. Another problem with very deep networks is the problems to train. DenseNet201 solves this issue since each layer has direct access to the gradients from the loss function and the original input image

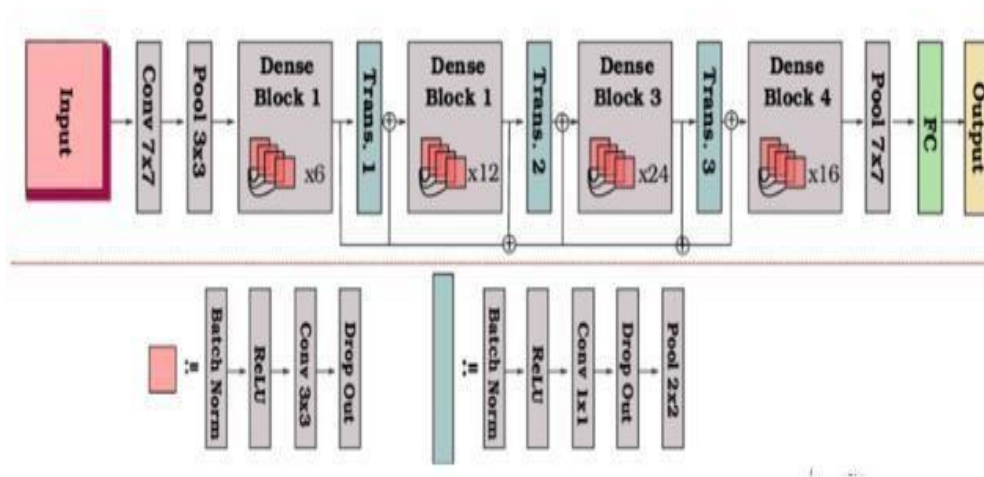


Figure 3.4 DenseNet201 Architecture Diagram [Source[2]]

3.7.1 Transition Layers

Since each dense block will increase the number of channels, adding too many layers will lead to an excessively complex model. A transition layer is used to control the complexity of the model. It reduces the number of channels by using a 1×1 convolutional layer. It uses the transition layer to halve the height and width and halve the number of channels, further reducing the complexity of the model.

3.8 XceptionNet

Xception by Google, stands for Extreme version of Inception, is reviewed with a modified depth wise separable convolution, it is even better than Inception-v3. The Xception module has 3 main parts, the Entry flow, the Middle flow and the Exit flow as shown in Fig 3.5. The entry flow has two blocks of convolutional layers followed by a ReLU activation. There are also various Separable convolutional layers. There are also Max Pooling layers.

When the strides are different than one, the strides are also mentioned.

3.8.1 Separable Layers

Separable convolutions consist of first performing a depth-wise spatial convolution (which acts on each input channel separately) followed by a pointwise convolution which mixes the resulting output channels

$$\text{Traditional Convolutional layer} = 3 \times 3 \times 3 \times 64 = 1,728$$

$$\text{Separable Convolutional layer} = (3 \times 3 \times 1 \times 3) + (1 \times 1 \times 3 \times 64) = 27 + 192 = 219$$

3.8.2 Model Parameter

The total parameters of the DenseNet201 architecture are 20,865,578. In this 4,098 of them are learnable parameters and 20,861,480 non-learnable parameters.

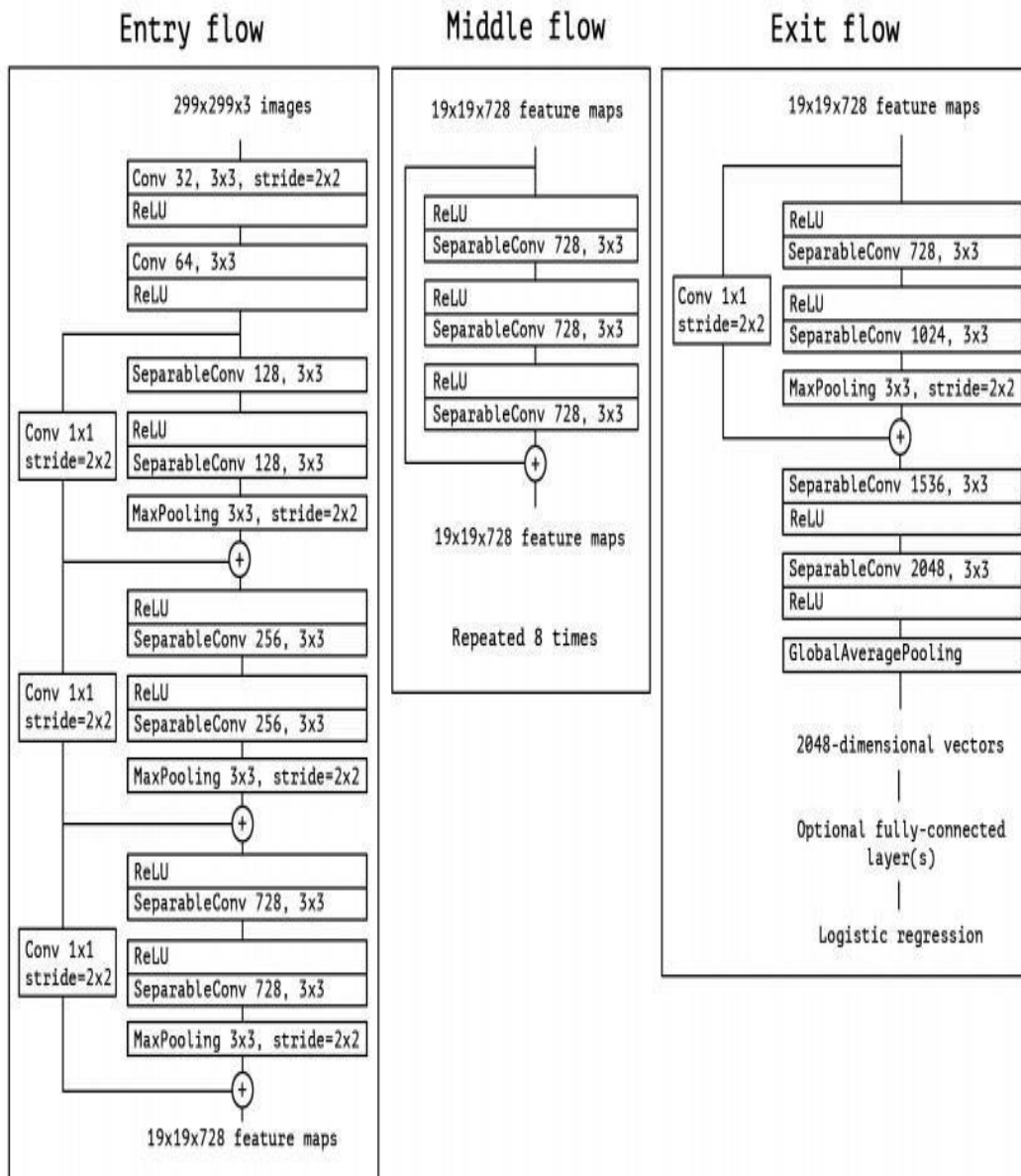


Figure 3.5 XceptionNet Architecture

3.9 Training

For training the CNN model, optimizer and early stopping are used. The parameters such as softmax Activation function, Adam optimizer, 5 epochs, 100 steps per epoch, Batch size of 32 and categorical cross entropy loss function were used in training the model. Early stopping is an optimization technique used to reduce over fitting without compromising on model accuracy. The main idea behind early stopping is to stop training before a model starts to over fit.

This proposed model uses an Adam optimizer, and this optimization is a stochastic gradient descent method that is based on adaptive estimation of the first-order and second-order moments. An epoch is a term used in deep learning and indicates the number of passes of the entire training dataset, the CNN algorithm has completed. The proposed system is trained up to 5 epochs for various models such as VGG19, DenseNet201, and XceptionNet. The best accuracy is found after 5 epochs. There is no variation of bias got after 5 epochs.

Batch size plays a major role in the CNN-based model. In this proposed method, the batch size is 32. The best accuracy was found using a batch size of 32. The batch mainly divides the training dataset and train model batch-wise respectively. Over fitting was avoided by using early stopping and regularizers in the first dense layer.

Thus, initially Dataset was splitted in the ratio of 67% and 33%. Augmentation technique is used to enhance the Chest X-Ray images to avoid overfitting. Three models such as VGG19, DenseNet201, XceptionNet Models were used to train and test the dataset where finally, the model performance evaluated using graph.

CHAPTER 4

RESULTS

The chest- X ray images are divided into Training set 3912 images, and testing set 1929. the images are resized to 224*244 size normalized before feeding them into the CNN networks. The CNN models used are VGG19, DenseNet201, XceptionNet. In figure 4.1 - 4.3, accuracy and loss graph of VGG19, DenseNet201, XceptionNet Models are shown.

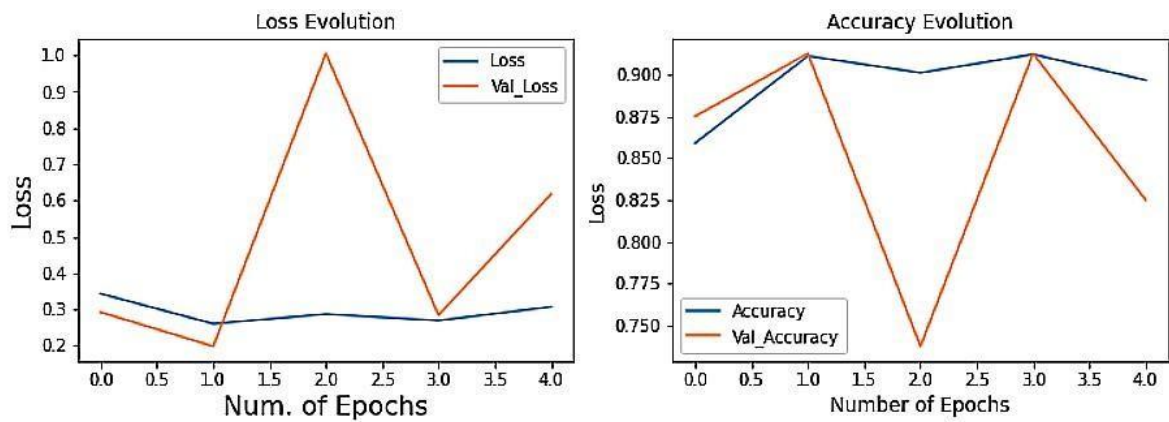


Figure 4.1 Accuracy and loss graph of VGG19

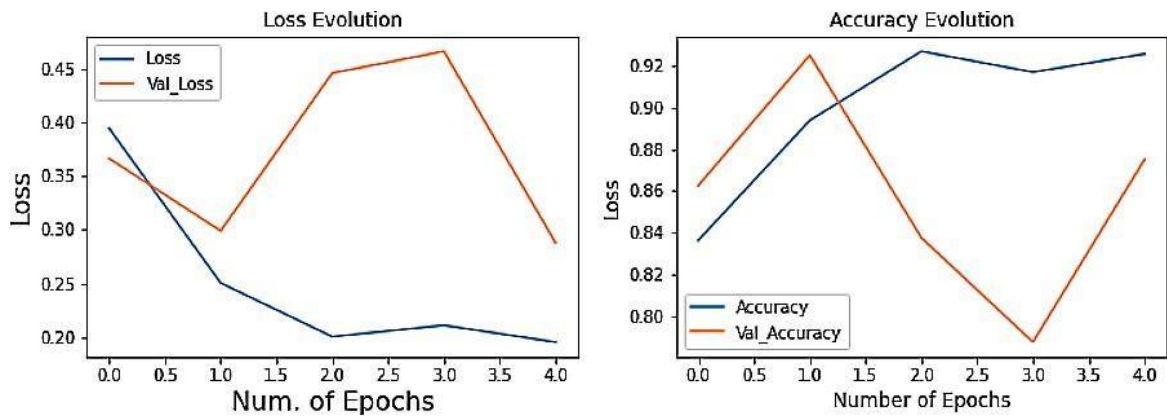


Figure 4.2 Accuracy and loss graph of DenseNet201

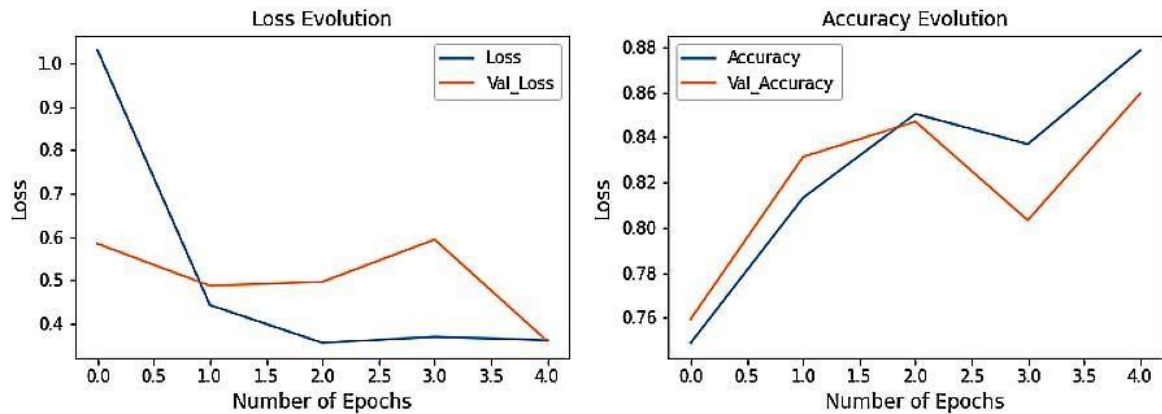


Figure 4.3 The accuracy and loss graph of XceptionNet The accuracy of the model assessed using Equation 4.1.

$$\frac{TP + TN}{TP + TN + FP + FN} \text{ ----- 4.1}$$

Where,

TP (True Positive) refers to Pneumonia Image being classified as Pneumonia.

TN (True Negative) refers to Normal Image being classsified as Normal.

FP (False Positive) refers to Normal Image as Pneumonia.

FN (False Negative) refers to Pneumonia Image as Normal.

Table 4.1 provides the value of test accuracy in VGG19, XceptionNet, DenseNet201 Models. These are trained with 5 epochs and batch size of 32. **Table**

4.1 Performance Evaluation

Model Name	Test Accuracy %
VGG19	84.70
XceptionNet	83.37
DenseNet201	91.49

The best accuracy value is obtained from DenseNet201 Model (91.49 %) which is good considering the size of data used.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This study describes a CNN-based model aiming to diagnose pneumonia on chest X-ray images. Data augmentation technique was used to enhance the chest X-ray images in the dataset. The three CNN architecture such as VGG19, DenseNet201, XceptionNet were used to extract the features from original images or previous feature maps, which contained only six layers combining ReLU activation function, drop operation, and max pooling layers. These convolutional neural networks models were successfully implemented by employing various methods of parameter tuning like adding dropout, changing learning rates, changing the batch size, number of epochs, adding more complex fully connected layers and changing various stochastic gradient optimizers.

5.2 Future Work

In the future, it is proposed to use a Raspberry Pi Board with a camera to develop a Pneumonia Classification expert system using Edge Machine Learning system. The web application will display the results of the Pneumonia Classification on the web page. This system will serve AI-based pneumonia classifier using Edge Impulse model.

REFERENCES

- [1] Alaa M. A. Barhoom and Prof. Dr. Samy S. Abu Naser, (2022), “Diagnosis of Pneumonia Using Deep Learning International Journal of Academic Engineering Research”, Vol. 6 (Issue 2), pp. 48-68.

- [2] Albawi, S.; Mohammed, T.A.; Al-Zawi, S, (2017), “Understanding of a convolutional neural network”, 2017 International Conference on Engineering and Technology (ICET), Antalya, Turkey, 21–23 August 2017, pp. 1–6.
- [3] Ali Sharif Razavian, Hossein Azizpour, Josephine Sullivan, and Stefan Carlsson, (2014), “CNN features off-the-shelf: an astounding baseline for recognition”, in proceedings of IEEE Conference on Computer Vision and Pattern Recognition workshops, pp. 806 - 813.
- [4] M. Anthopoulos, S. Christodoulidis, L. Ebner, A. Christe, and S.Mougiakakou, (2016), "Lung Pattern Classification for Interstitial Lung Diseases Using a Deep Convolutional Neural Network," in IEEE Transactions on Medical Imaging, vol. 35, (Issue no. 5), pp. 1207-1216.

: SAMPLE CODE

Xception Net Model

```

image_size = 224
BATCH_SIZE = 32 n_labels
= 2 gen =
ImageDataGenerator(
rescale=1/255.,
horizontal_flip=True,
vertical_flip=True,
rotation_range=0.4,
zoom_range=0.4)
Train_gen =
gen.flow_from_directory("/content/drive/MyDrive/chest
xray/train", target_size=(image_size, image_size),
batch_size= BATCH_SIZE) Test_gen =
gen.flow_from_directory("/content/drive/MyDrive/chest
xray/test", target_size=(image_size,
image_size),batch_size=BAT CH_SIZE) base_model =

```

```

tf.keras.applications.Xception(weights="imagenet",
input_shape=(224,
224, 3), include_top=False
base_model.trainable = False inputs =
base_model.input x =
tf.keras.layers.GlobalAveragePooling2D()(base_model.output) x
= tf.keras.layers.Dropout(0.2)(x) outputs =
tf.keras.layers.Dense(2)(x) model = tf.keras.Model(inputs,
outputs) model.summary() layers = model.layers

Printf("The model has {len(layers)} layers")//The model has
135 layers
model.compile(optimizer='adam',loss='categorical_crossentropy'
,m etrics=['accuracy'])

history = model.fit_generator( Train_gen,
steps_per_epoch=100,ep ochs=5,validation_data=Test_gen,
validation_steps=10)

```

VGG19 Model

```

from keras.layers import Input, Lambda, Dense, Flatten
from keras.models import Model from
keras.applications.VGG19 import VGG19 from
keras.applications.VGG19 import preprocess_input from
keras.preprocessing import image from
keras.preprocessing.image import ImageDataGenerator
from keras.models import Sequential import tensorflow
as tf from keras.models import load_model import numpy
as np from glob import glob import matplotlib.pyplot
as plt IMAGE_SIZE = 224 batchSize = 32 NUM_CLASSES =
2 gen = ImageDataGenerator( rescale=1/255.,
horizontal_flip=True, vertical_flip=True,
rotation_range=0.4, zoom_range=0.4) Train_gen =

```

```

gen.flow_from_directory("/content/drive/MyDrive/chest_xray/train", target_size=(224, 224),batch_size=10)
Test_gen = gen.flow_from_directory(
"/content/drive/MyDrive/chest_xray/test",target_size=(224,
224),batch_size=8)
vgg =
VGG19(weights='imagenet',include_top=False,input_shape=(22
4,224,3))
for layer in vgg.layers: layer.trainable =
False x = Flatten()(vgg.output)
predictions = Dense(2,activation='softmax')(x)
model = Model(inputs=vgg.input, outputs=predictions)
model.summary() model.compile(optimizer='adam',
loss='categorical_crossentropy', metrics=['accuracy'])
history = model.fit_generator(Train_gen,
steps_per_epoch=100,epochs=5,validation_data=Test_gen,
validation_steps=10)

```

DenseNet201 Model

```

import tensorflow as tf from
tensorflow.keras.preprocessing.image import
ImageDataGenerator from keras.applications.densenet import
DenseNet201 from keras.layers import Dense,
GlobalAveragePooling2D from keras.models import Model from
keras import backend as K gen = ImageDataGenerator(
rescale=1/255., horizontal_flip=True,
vertical_flip=True, rotation_range=0.4,
zoom_range=0.4) Train_gen =
gen.flow_from_directory("/content/drive/MyDrive/chest_xray/train", target_size=(224, 224),batch_size=32)
Test_gen = gen.flow_from_directory(
"/content/drive/MyDrive/chest_xray/test",target_size=(224,
224),batch_size=32) model_densenet =
tf.keras.applications.DenseNet201(input_shape=

```

```
IMAGE_SIZE,include_top=False,weights='imagenet',pooling='avg'  
) model_densetnet.trainable = False input =  
model_densetnet.input # Rebuild top x =  
tf.keras.layers.Dense(128,activation="relu")(model_densetnet  
.output)
```